

Cryptography Laboratory Report: DES Modes of Operation and Key Exhaustion

Christopher Alejandro Ortiz Bustillos

¹Cryptography Course, Semester 2, 2026, Ibarra, Imbabura, Ecuador

Corresponding author: Christopher Alejandro Ortiz Bustillos.

ABSTRACT This report details the implementation and analysis of the Data Encryption Standard (DES). It examines the Electronic Codebook (ECB) and Cipher Block Chaining (CBC) modes of operation, analyzing their behavior with repeated plaintext blocks and their error propagation characteristics. Furthermore, the report explores a parallelized brute-force attack on a reduced DES key space, estimating the computational feasibility of an exhaustive search on the complete 56-bit key space.

INDEX TERMS Brute-force attack, Cipher Block Chaining (CBC), Cryptography, Data Encryption Standard (DES), Electronic Codebook (ECB), Parallel Computing.

I. Introduction

THE objective of this laboratory is to evaluate the security and operational characteristics of the Data Encryption Standard (DES). Despite being deprecated due to its short 56-bit key length, DES provides a foundational understanding of symmetric block ciphers. This laboratory focuses on two critical aspects: the impact of modes of operation (ECB and CBC) on data confidentiality, particularly concerning pattern leakage and error propagation, and the vulnerability of the cipher to brute-force attacks via parallel computing.

II. Modes of Operation

Block ciphers process data in fixed-size blocks (8 bytes for DES). To encrypt messages of arbitrary length, modes of operation are utilized. PKCS#7 padding is applied to ensure the plaintext length is a multiple of the block size.

A. Electronic Codebook (ECB)

ECB is the simplest mode, where each plaintext block is encrypted independently using the same key: $C_i = E(K, P_i)$. While highly parallelizable, ECB fails to hide data patterns, as identical plaintext blocks produce identical ciphertext blocks.

B. Cipher Block Chaining (CBC)

CBC randomizes each block by XORing the plaintext with the previous ciphertext block before encryption: $C_i = E(K, P_i \oplus C_{i-1})$. The first block utilizes an Initialization

Vector (IV): $C_1 = E(K, P_1 \oplus IV)$. This chaining mechanism ensures that identical plaintext blocks yield distinct ciphertext blocks, providing semantic security against chosen-plaintext attacks.

III. ECB versus CBC Experiment

To demonstrate the vulnerabilities of ECB, a repeated-block experiment was conducted. As documented in `modes-test.txt`, the plaintext consists of the 8-byte sequence 4142434445464748 repeated four times.

Using the key 133457799bbcdff1, the ECB ciphertext clearly reflects the plaintext structure. Blocks 1 through 4 are identical: 56467e175662aff6. Block 5 corresponds to the PKCS#7 padding (640907ba2fde085c).

In contrast, the CBC mode (using IV 0123456789abcdef) produces completely pseudo-random blocks, completely masking the repetition. The resulting blocks are:

- Block 1: 72ab28e79136b967
- Block 2: 0d2be1c34bb3d425
- Block 3: c5ff2418fee0dd01
- Block 4: 22e41521a7146b1c

Furthermore, modifying the IV to an alternative value (fedcba9876543210) completely changes the resulting CBC ciphertext starting from Block 1 (7c558e391d01467d), demonstrating the avalanche effect introduced by the IV chaining.

IV. Error Propagation

Transmission errors affect modes of operation differently. In this experiment, a corruption was introduced by XORing the first byte of the ciphertext with `0x15` (`modes-test.txt`).

A. ECB Error Propagation

In ECB mode, the corruption alters only the specific ciphertext block. Block 1 changes from `56467e175662aff6` to `43467e175662aff6`. During decryption, this will completely randomize the 8 bytes of the first plaintext block, but Blocks 2 through 5 will decrypt flawlessly. The error does not propagate.

B. CBC Error Propagation

In CBC mode, the ciphertext Block 1 is corrupted from `72ab28e79136b967` to `67ab28e79136b967`. During decryption, this causes two distinct effects:

- 1) **Complete scrambling:** Plaintext Block 1 will be completely randomized because it passes through the non-linear decryption function $D(K, C'_1)$.
- 2) **Targeted bit-flipping:** Plaintext Block 2 will have exact bits flipped corresponding to the `0x15` mask, because the corrupted C'_1 is XORed with the properly decrypted output of C_2 . Subsequent blocks (Block 3 onwards) remain entirely unaffected.

V. Brute-Force Method

A brute-force exhaustive search attacks the symmetric key space by systematically checking all possible combinations. Given DES uses a 56-bit effective key, the full key space is $2^{56} \approx 7.2 \times 10^{16}$ keys.

To make the experiment practically feasible in a laboratory setting, a reduced key-space construction is implemented. By fixing N bits of the key to known correct values, the search space is reduced to 2^{56-N} . The algorithm iterates through all combinations of the unknown bits, encrypts a known plaintext, and compares the output with the target ciphertext.

For this laboratory, a reduced search space of 1,048,576 keys (2^{20}) was evaluated. To establish a reliable baseline for sequential performance, the exhaustive search was executed using a single CPU worker. This sequential test was repeated three times to account for system background processes and CPU scheduling variations, yielding an average execution time and throughput. The results of these baseline iterations are detailed in Table 1.

TABLE 1. Sequential Execution Performance (1 Worker)

Attempt	Time [s]	Keys Tested	Throughput [keys/s]
1	299.4736	1048576	3501.40
2	292.2436	1048576	3588.02
3	368.2799	1048576	2847.23
Average	319.9990	1048576	3276.81

VI. Parallel Implementation

To accelerate the brute-force search, the key space is partitioned into chunks and distributed among multiple CPU workers using parallel processing (e.g., Python's `multiprocessing` library). The total reduced key space S is divided into W subsets, where W is the number of available logical CPU cores. Each worker receives a defined start and end key index, processing its subspace independently. Once a worker finds a key that produces a match, it signals the parent process to terminate all other workers.

VII. Experimental Results for Parallel Implementation

The execution time was measured for sequential and parallel executions over a reduced search space of 1,048,576 keys (2^{20}). For the sequential execution, the experiment was repeated three times, yielding an average execution time of 319.9990 seconds.

The parallel performance metrics are defined as follows:

- **Workers (W):** 1, 2, 4, and 6 cores.
- **Execution Time (T_w):** The time taken to explore the reduced space.
- **Throughput:** Measured in keys evaluated per second (Keys/s).
- **Speedup (S_p):** Calculated as $S_p = \frac{T_1}{T_p}$.
- **Parallel Efficiency:** Calculated as $E_p = \frac{S_p}{W}$.

TABLE 2. Parallel Performance Metrics for DES Brute-Force Attack

Workers	Time [s]	Keys Tested	Keys/s	S_p	E_p
1	319.9990	1048576	3276.81	1.0000	1.0000
2	176.7616	1048576	5932.15	1.8103	0.9052
4	83.0287	1048576	12629.07	3.8541	0.9635
6	62.9754	1048576	16650.57	5.0813	0.8469

The results show a near-linear speedup as the number of CPU workers increases, confirming that key exhaustion is an “embarrassingly parallel” problem. However, a slight drop in parallel efficiency can be observed at 6 workers (84.69%), caused by the computational overhead of process creation and synchronization.

Figure 1 illustrates the steady increase in keys evaluated per second as cores are added, while Figure 2 compares the actual measured speedup against the theoretical ideal linear speedup.

VIII. DES-56 Extrapolation

Using the maximum throughput achieved by the parallel implementation (6 workers), we can extrapolate the expected time to brute-force the entire 2^{56} DES key space. If the maximum throughput is $R = 16650.57$ keys/second, the estimated exhaustive-search time ($T_{\max}$) is:

$$T_{\max} = \frac{2^{56}}{R} \approx 4.33 \times 10^{12} \text{ seconds}$$

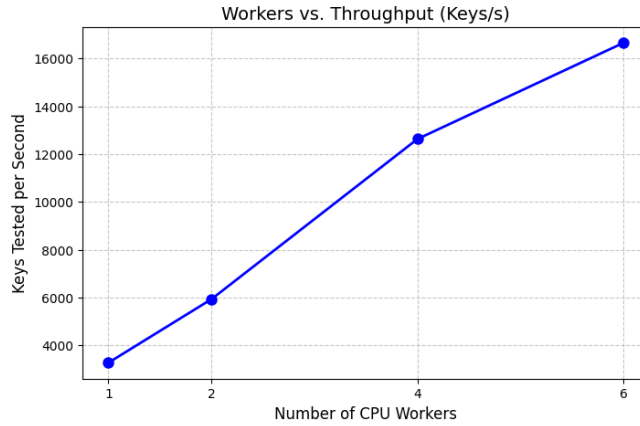


FIGURE 1. Throughput (Keys/s) as a function of the number of CPU workers.

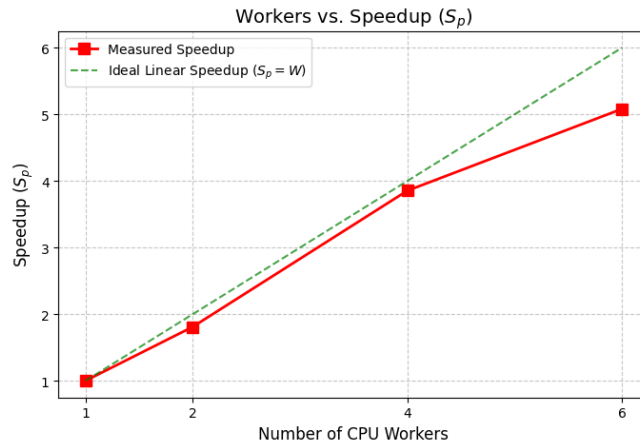


FIGURE 2. Measured speedup (S_p) compared to ideal linear speedup.

This translates to roughly 1.20×10^9 hours, or 1.37×10^5 years. The average expected key-recovery time (T_{avg}) is approximately 68,600 years. While these numbers seem large for a Python script, they clearly demonstrate why 56 bits is considered entirely insecure against modern optimized hardware (GPUs/ASICs) which can process billions of keys per second.

IX. Security Discussion

Based on the experimental evidence obtained in the laboratory, the following security implications are addressed:

1) Why does a correct implementation of DES not imply that DES is secure today?

A correct implementation only ensures that the algorithm operates exactly according to the mathematical specifications of the standard. However, security relies on resistance to attacks. Regardless of how well it is coded, DES is inherently insecure today because its 56-bit key length is too small to withstand brute-force attacks from modern computational hardware.

2) Which DES property makes exhaustive search practical compared with modern symmetric ciphers?

Its short effective key length of 56 bits. A search space of 2^{56} combinations is small enough to be exhaustively computed by distributed systems or specialized hardware (like FPGAs) in a matter of days or hours, unlike modern ciphers that use 128-bit or 256-bit keys.

3) How does adding CPU cores affect brute-force performance?

Adding CPU cores divides the total search space into smaller chunks processed concurrently. This drastically reduces the total execution time and increases the throughput (keys tested per second), yielding a near-linear speedup, as seen when scaling from 1 worker to 6 workers.

4) Why does parallel efficiency generally decrease as the number of workers increases?

Efficiency drops due to the overhead associated with parallel computing. Creating processes, managing inter-process communication (IPC), and synchronizing tasks (such as checking a shared stop event) consume CPU cycles and memory. These delays prevent the speedup from scaling perfectly (e.g., 6 workers resulted in a 5.08x speedup, not 6.0x).

5) Would replacing DES with AES-128 make the same exhaustive-search strategy practical? Explain quantitatively.

No. AES-128 has a key space of 2^{128} . Using the best measured throughput of $R = 16650.57$ keys/s, the expected time would be:

$$T_{max} = \frac{2^{128}}{16650.57} \approx 2.04 \times 10^{34} \text{ seconds}$$

This equates to trillions of times the estimated age of the universe, making the brute-force strategy quantitatively and practically impossible.

6) Does CBC increase the size of the DES key space?

No. The Cipher Block Chaining (CBC) mode changes the algorithm's operational structure by XORing the plaintext with the previous ciphertext block (or IV). However, the underlying encryption function still strictly uses a 56-bit key. Therefore, the key space remains exactly 2^{56} .

X. Conclusions

This laboratory demonstrated the operational mechanics of DES and its standard modes of operation. Key findings include:

- 1) ECB mode fails to mask data patterns, confirming its vulnerability to structural cryptanalysis.
- 2) CBC mode effectively masks patterns but exhibits specific error propagation characteristics that allow for targeted bit-flipping if integrity checks are absent.
- 3) A parallelized brute-force attack exhibits near-linear scalability, proving that a 56-bit key length is triv-

ial to exhaust with modern or distributed computing resources. Although scripting languages yield higher absolute times, the relatively small 2^{56} search space mathematically guarantees obsolescence against modern architecture.